Overview

The assembly of datasets in the `ptirtools` library is handles by the `assembly` module. A procedure is defined declaratively, by passing a sequence of `Operation` objects into an `AssemblyProcedure`:

```
# directly in the constructor
procedure = AssemblyProcedure( Operation1, Operation2, Operation3, ... )

# add more steps to the object
procedure << NewLastOperation1 << NewLastOperation2
NewFirstOperation1 >> NewFirstOperation2 >> procedure
```

An `Assembler` is created as the object to manage stepping through the procedure:

```
assembler = Assembler(procedure, verbose=False)
```

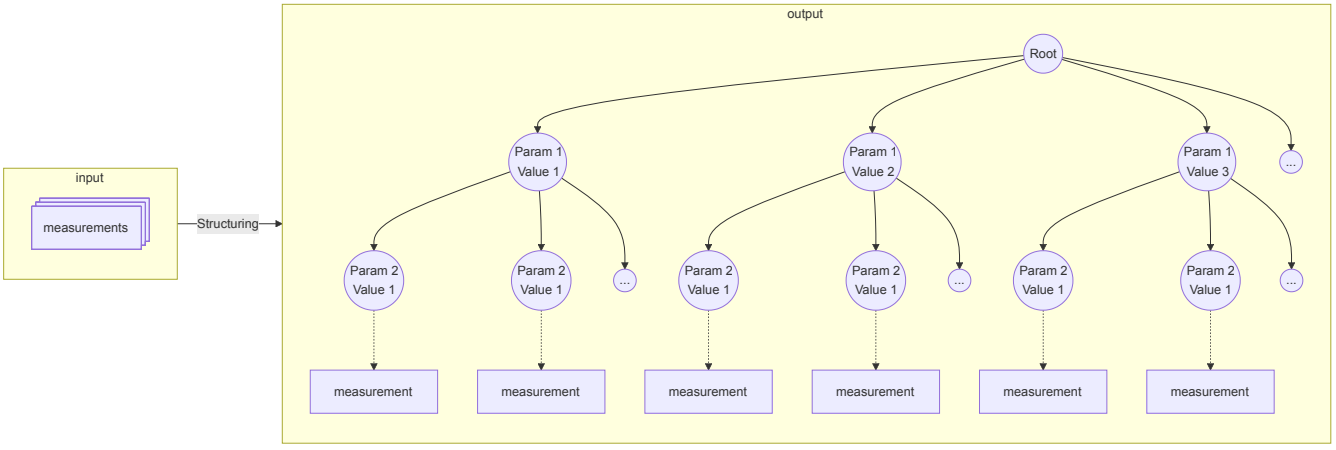
The procedure is invoked through a method of the assembler:

```
dataset = assembler.assemble(measurements)
```

The assembly of ptir datasets is done in two broad steps: First, the unordered collection of measurements is **structured**, then, from that structure, the dataset is **assembled**.

Structuring

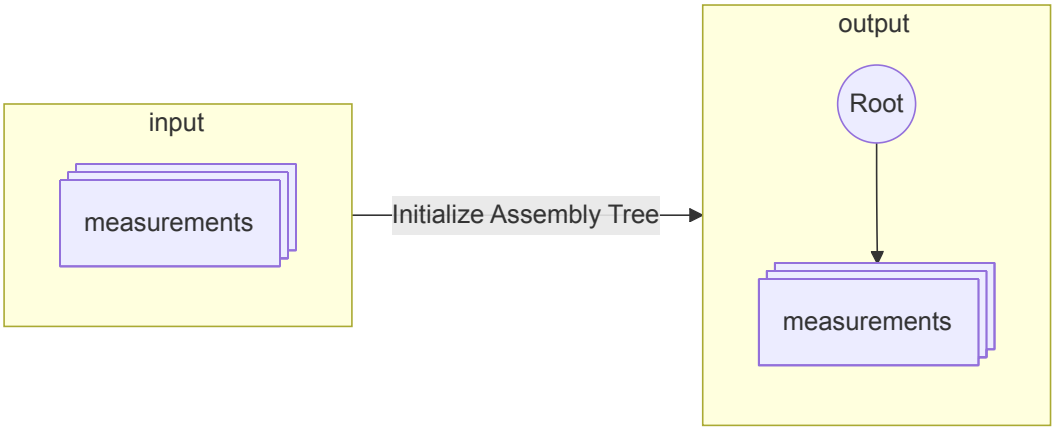
The goal is to turn an unordered set of measurement objects into a tree whose leaves are single measurement objects:



To do that, we can perform a few kinds of operations:

Creating a Root Node

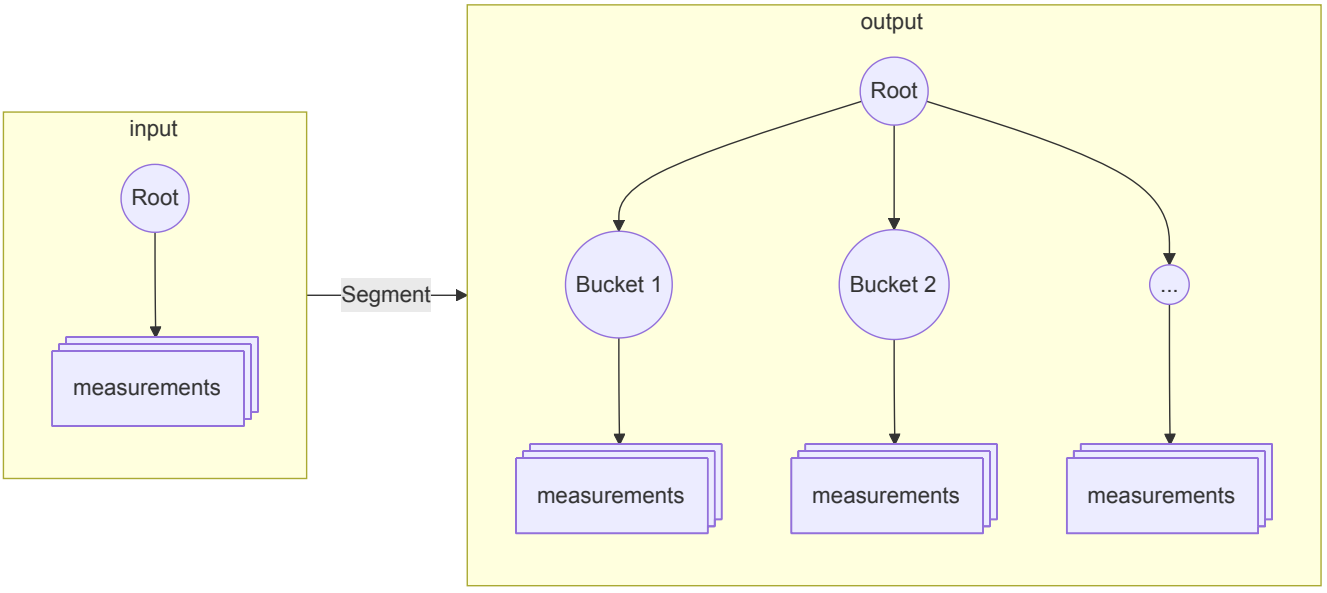
This barely counts as an operation but creates the starting configuration that we'll get back to when stepping through the assembly process:



Now, there is a node that points with a collection of measurements assigned to it.

Segmentation

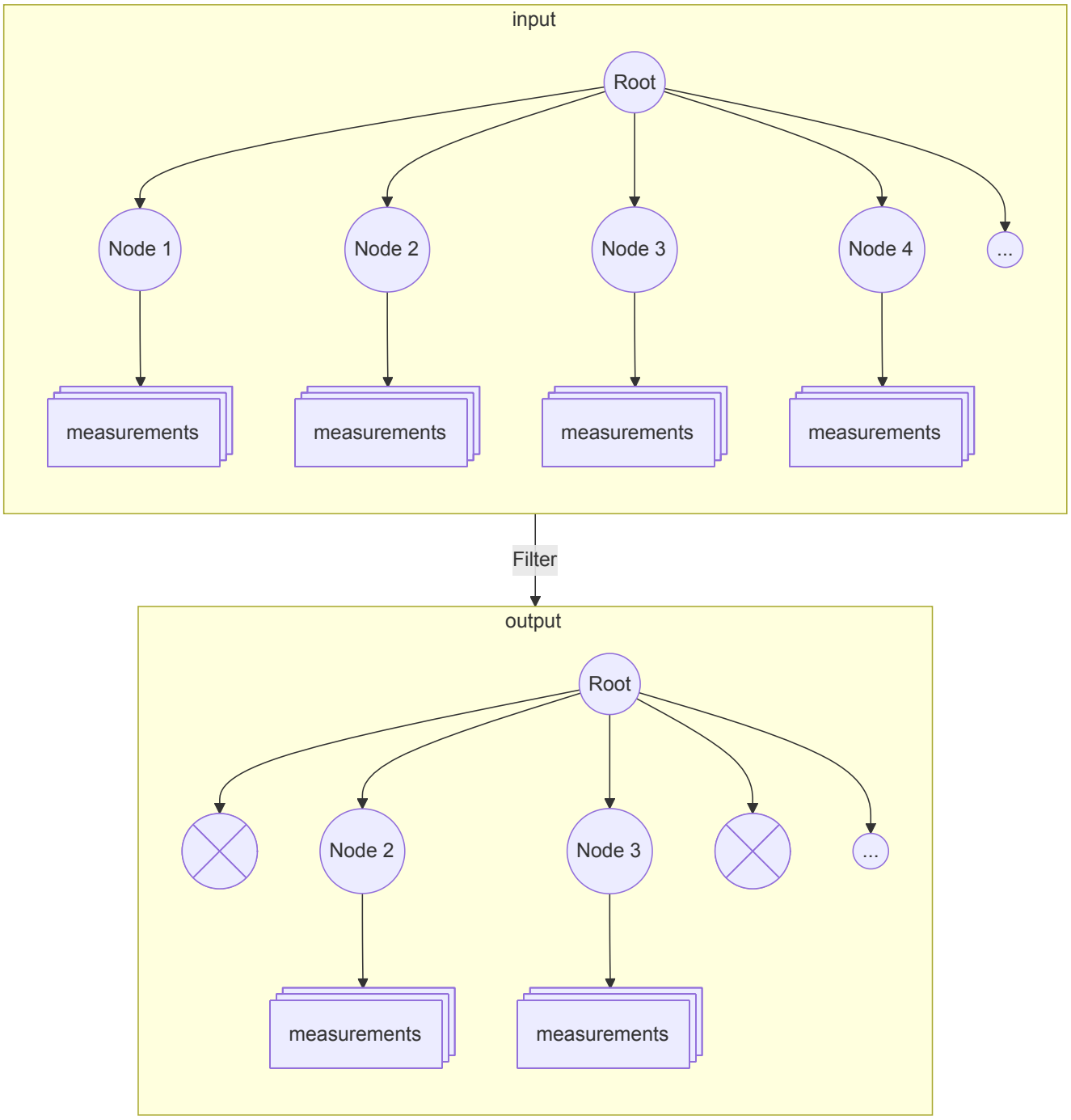
Sort measurements into buckets depending on some parameter and label the buckets with a representative value of the parameter.



Note that there is no categorical reasons that the segmented collections should be pairwise disjoint. Using the standard Segmentation Operation, I expect this will always be the case, but one could implement a segmentation operation that sorts a measurement into multiple buckets.

Filtering

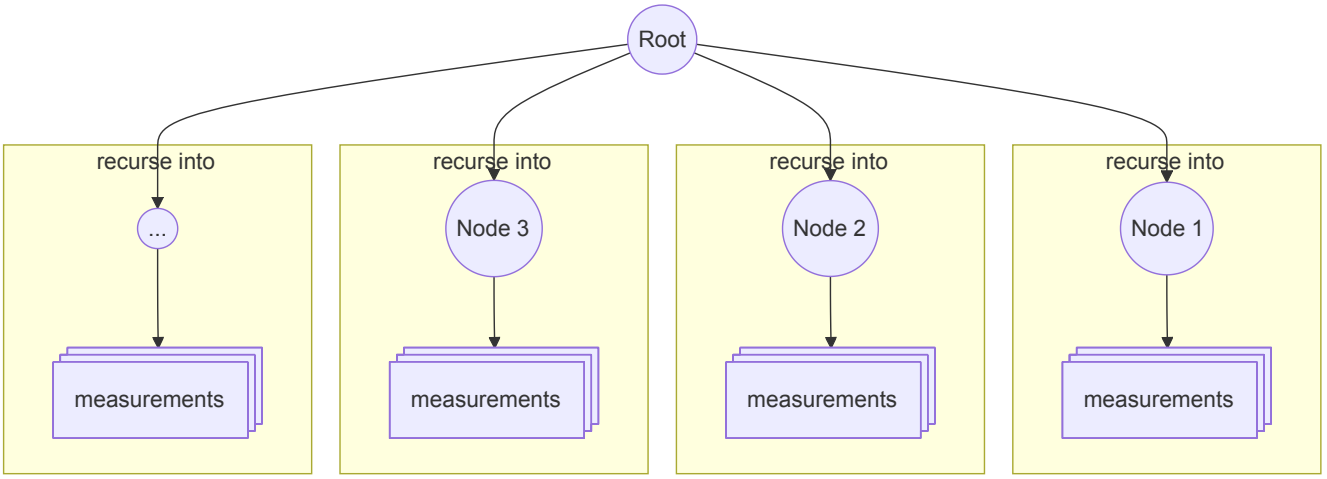
Discard buckets depending on some condition regarding the bucket label.



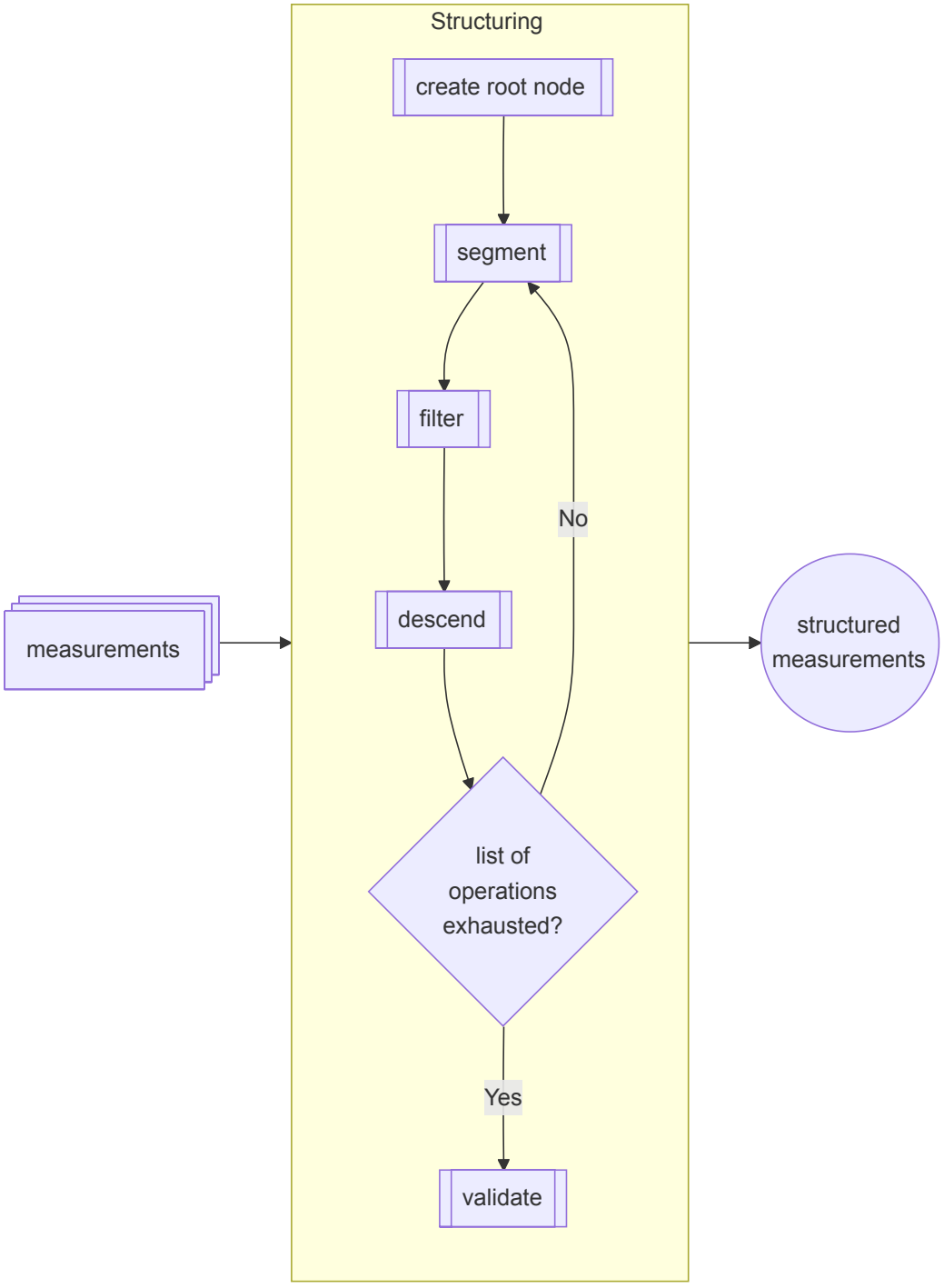
A special case of Filtering is **Selection**, where exactly one bucket remains.

Descent

To continue the Structuring, we will iterate over the subordinate nodes, re-running these steps on each of them. But we also need to know what to do with the nodes on the way back up. We so we need to first tag the Root node here with information



These recursive steps will be repeated until the list of instructions is exhausted. Finally, we have to assert that each leaf node points to a collection of measurements with exactly one element:



Note that the execution structure doesn't need to be quite as rigid as shown here. Conditions may warrant a change of the order in maybe just one place. Or the execution of multiple consecutive filter operations. Or none between a Segmentation and a Descent. At any point it may be helpful for the user to declare an assertion. These should be checked, depending on what they are exactly on the way down or back up or both. All of these should be declared by the user explicitly. In the interest of customizability and a powerful tool, the possibility of errors due to the user supplying an invalid structure is acceptable.

User-side implementation should look something like this:

```
procedure = AssemblyProcedure(
    Segmentation(top_level_parameter), Select('<exact expected value>'),
    Descent(),
    Segmentation(next_parameter), Filter(condition_1), Filter(condition_2), AssertUnique(),
    Descent(),
    Segmentation(another_parameter, tolerance=param_3_tolerance), AssertNotUnique(), Descent(),
    # and so on...
)
```

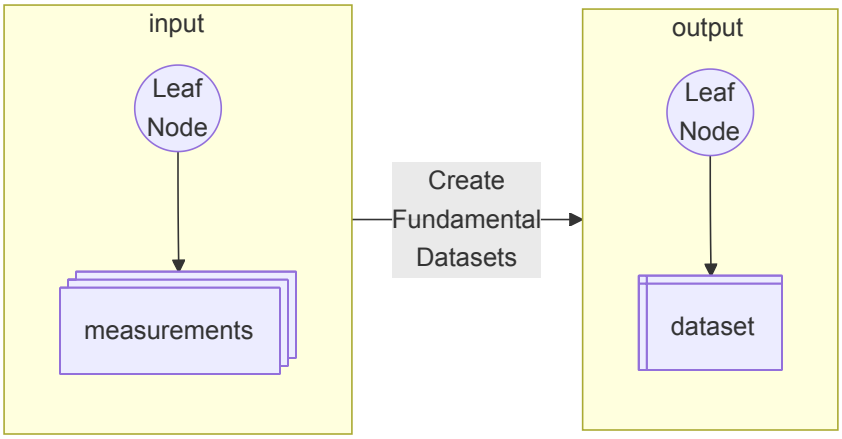
In this example, the `Descent` operation is kept deliberately vague, as it needs to perform some more steps that are not relevant to the structuring step per se, but to the next step:

Assembly

With the Structuring complete, we have built a tree tree structure for our measurements from the root to the leaves. Now, to create the dataset, we need to collapse the tree again, this time from the leaves to the root. To that end, we perform the following kinds of operations

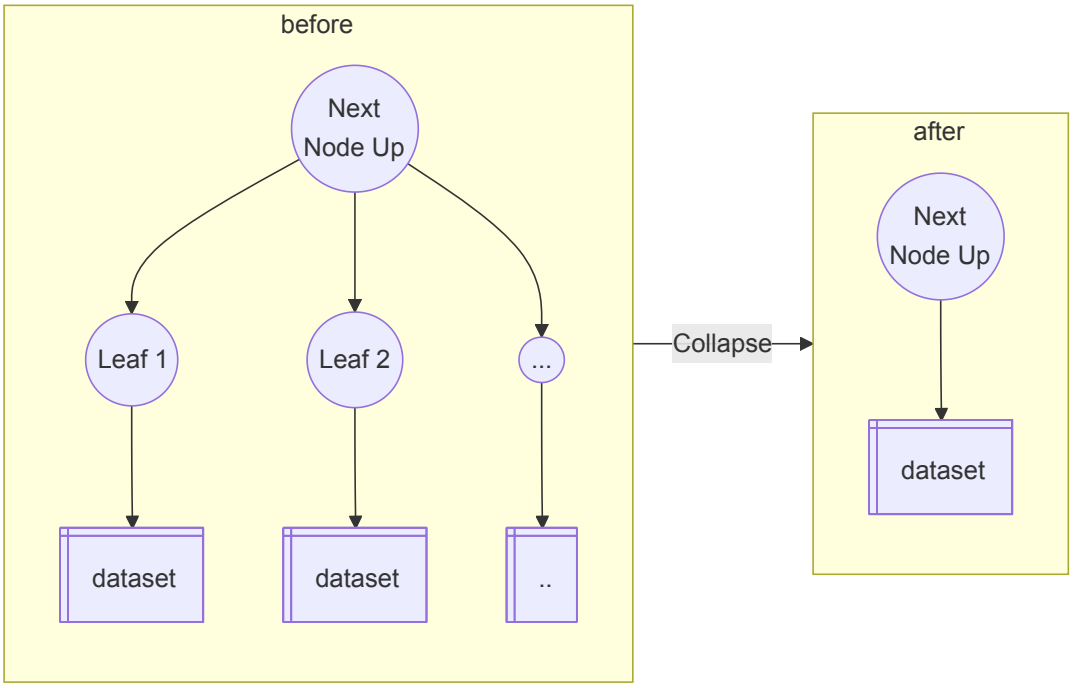
Creation of Fundamental Datasets

This needs to be done once at each leaf: From the measurement objects, we must create a more generic object that we can arbitrarily add axes to and concatenate together:



This process is not quite trivial: The shape and axes of the dataset do already depend on the kind of measurement we have on hand; as do any data conversions that might have to be done.

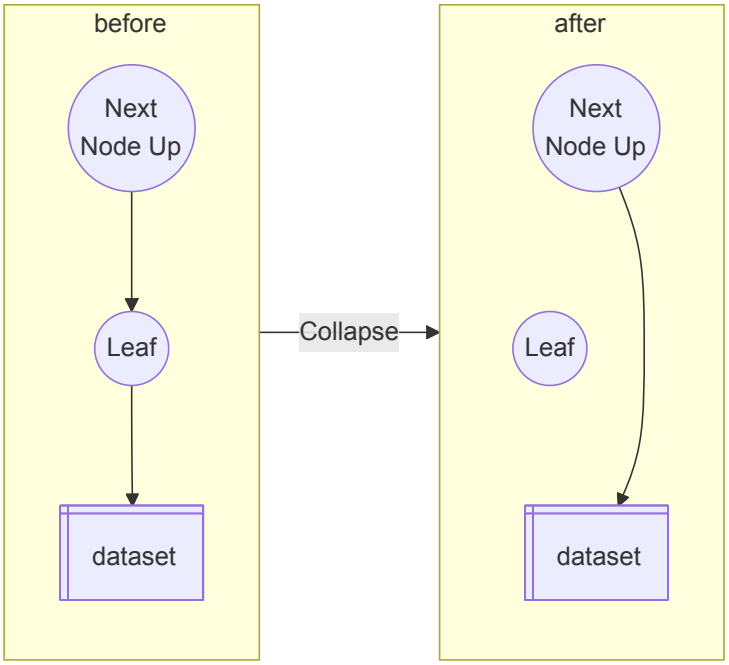
Now we traverse the tree backwards from the leaves to the root. For each step of walking back up, the structure will look like this:



This collapse can be done in a few different ways. They fall in two broad categories, depending on whether there is only one leaf node (`AssertUnique`) or there are multiple.

Exactly One Leaf Node

In this case, the fundamental operation will always be very simply described by attaching the dataset to the next node up:



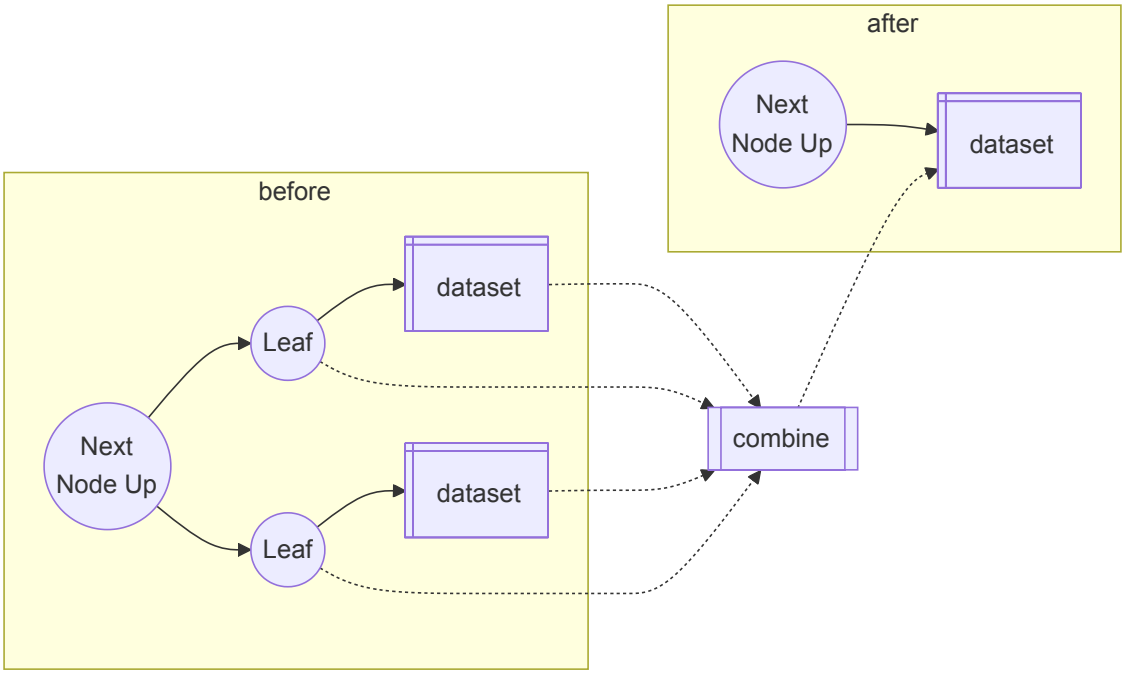
But, what we've ignored so far, is that the leaf node has some information attached to it: the parameter that was segmented by and its representative value for this subtree. There are different options what to do with this information.

- The trivial one is to simply discard it. (should maybe be the default behaviour of `Descent` with no further specification)
- Instead, the information could be attached to the dataset and propagated up the tree along with it. In this case, there needs to be a rule for the combination of possibly different values at higher levels. (use `TrackParameter` for this?)

More involved operations exist if there are multiple leaves whose datasets need to be assembled into one:

Arbitrary Number of Leaf Nodes

Here, an arbitrary number of datasets must be combined into one, using information attached to the corresponding leaf nodes (from the segmentation that created them) to define how exactly to combine / assemble them.



Parameter Transformation

Sometimes, it can be useful to have the segmentation parameters available in a different form. Operations should be available to do that before combination / assembly.

There should be one Operation to apply a function to each segmentation parameter value individually (this is useful for transforming the quite unwieldy specification of the OPTIR channel into a nice string-type key) and one to apply a function to the entire set of

parameter values simultaneously (useful for turning pairs of (x,y) lateral positions into a 1-dimensional spatial parameter (s) parametrizing the length along a line profile)

Parameter Transformations should substitute the original parameter specification with a new, user-defined one.

Data Transformation

Similarly, we might have to pass the dataset itself through some function, depending on the segmentation parameter value. While technically, it is possible to implement this using only Parameter Transformations and Accumulations, we're going to have this option too, for semantic clarity.

Broadly, datasets can be either **concatenated** or **accumulated**.

Concatenation

In concatenation, a dimension is added to the dataset, representing the segmentation parameter for this layer. Properties of the parameter or of the dataset w.r.t the parameter define the specific behaviour:

- If the dataset is homogeneous under this segmentation, datasets that are multidimensional arrays will stack properly. (Example: assembly of images into z-stacks or hyperspectrals, spectra into line profiles)
- If the parameter is quantitative / numeric, the dataset should be sorted along this axis. (Sort differently shaped high-dimensional stacks by modulation frequency)

The recipe for building a line profile from individual spectra might look something like this:

```
procedure = AssemblyProcedure(
    # work discard all measurements that don't have tpye "OPTIRSpectrum"
    Segment( type_parameter ), Select( "OPTIRSpectrum" ), AssertUnique(), Descend(),

    # Make buckets of spectra that were recorded within 40 µm of each other
    Segment( lateral_position_parameter, tolerance=40.0 ),
    # do the following steps on all of them
    Parametrize( is_homogeneous=False ),

    # within each bucket...
    # Separate spectra by lateral position
    Segment( lateral_position_parameter, tolerance=0 ),
    # turn the lateral position specification into a one-dimensional parameter specific to each line profile
    TransformParameterSpace(
        dummy_parameter = ParameterSpecification( "__dummy__",
                                                  is_quantitative = True,
                                                  name = "lateral position",
                                                  symbol = 's', unit = 'µm',
                                                  latex_symbol='s',
                                                  latex_unit=r"\mathrm{\mu m}"),
        function = lambda points : (
            np.sqrt( (p[0]-points[0][0])**2 + (p[1]-points[0][1])**2 ) for p in points
        )
    ),
    # turn it into an axis
    Parametrize( is_homogeneous=True )

    # ... stuff to do after (closer to the leaves of the tree)
```

Accumulation

In accumulation, datasets are combined into one of the same dimensionality through a function. Underlying datasets do not, in principle, need to have the same shape, but the responsibility of ensuring the accumulation is valid falls on the user.

A simple example use case is averaging over the measurements.

The accumulating function is passed a list of pairs of segmentation parameter and measurement. This way, parameters (especially qualitative / categorical ones) may be used to give the correct argument order for not pairwise-commutative accumulation functions.

An example use case for this is calculating the complex representation of the OPTIR signal. The user-side declaration should look something like this:

```
procedure = AssemblyProcedure(
    ### stuff to do before (closer to the root of the tree)

    # segment by optir channel
    Segment( optir_channel_parameter ),

    # replace optir channel specification with nice string-type keys
    TransformParameter(
        dummy_parameter = ParameterSpecification("__dummy__", <...>, "channel"),
        transformation = ( lambda channel : 'DC' if channel.harmonic_order == 0 else ... ),
    ),

    # check required channels exist
    AssertParameterValuesExist( ['DC','O-PTIR','Phase'] ),

    # calculate the relative optir response from the signal components
    Accumulate(
        accumulation_function = lambda dc, optir, phase : optir / dc * np.exp(1j*phase),
        argument_order = ( 'DC', 'O-PTIR', 'Phase' ),
    ),

    # Accumulation is a Descent Operation, so we are one layer further in now,
    # working on the individual signal components

    # segment by optir channel again
    Segment(optir_channel_parameter),

    # from the optir channel specification, determine if
    # this is a phase channel that needs unit correction
    TransformParameter(
        dummy_parameter = ParameterSpecification("__dummy__", <...>, "degree_phase_channel"),
         transformation = ( lambda channel : True if channel.signal_component == PHAS and channel.unit ==
'deg' else 0 ),
    ),

    # if we have a phase signal in degrees on hand, transform to radians.
    # otherwise do nothing.
    TransformData(
        function = lambda parameter_value, data : np.radians(data) if parameter_value else data
    )

    # we assume there is only one kind of channel here and now we don't
    # need that information anymore so we can discard it on the way back up
    AssertUnique(), Descend(),

    ### stuff to do after (closer to the leaves of the tree)
)
```

`Accumulate` should have subclasses for commonly used ways of collapsing an array along one dimension (mean, median, min, max, std, …)

Order of Declaration vs. Order of Execution

We have introduced a lot of steps and operations now and it may not be clear which step is executed when and in what order they need to be declared.

For the declaration order, the rule of thumb should be: Operations are declared from root to leaves.

As for the execution order, of course operations are performed when the execution is in that particular layer of the structure. However, some operations take effect during structuring (down/in), some during assembly (up/out).

Pure Structuring Operations

- Segmentation
- Parameter Transformation
  - TransformParameter
  - TransformParameterSpace
- Filtering / Selection

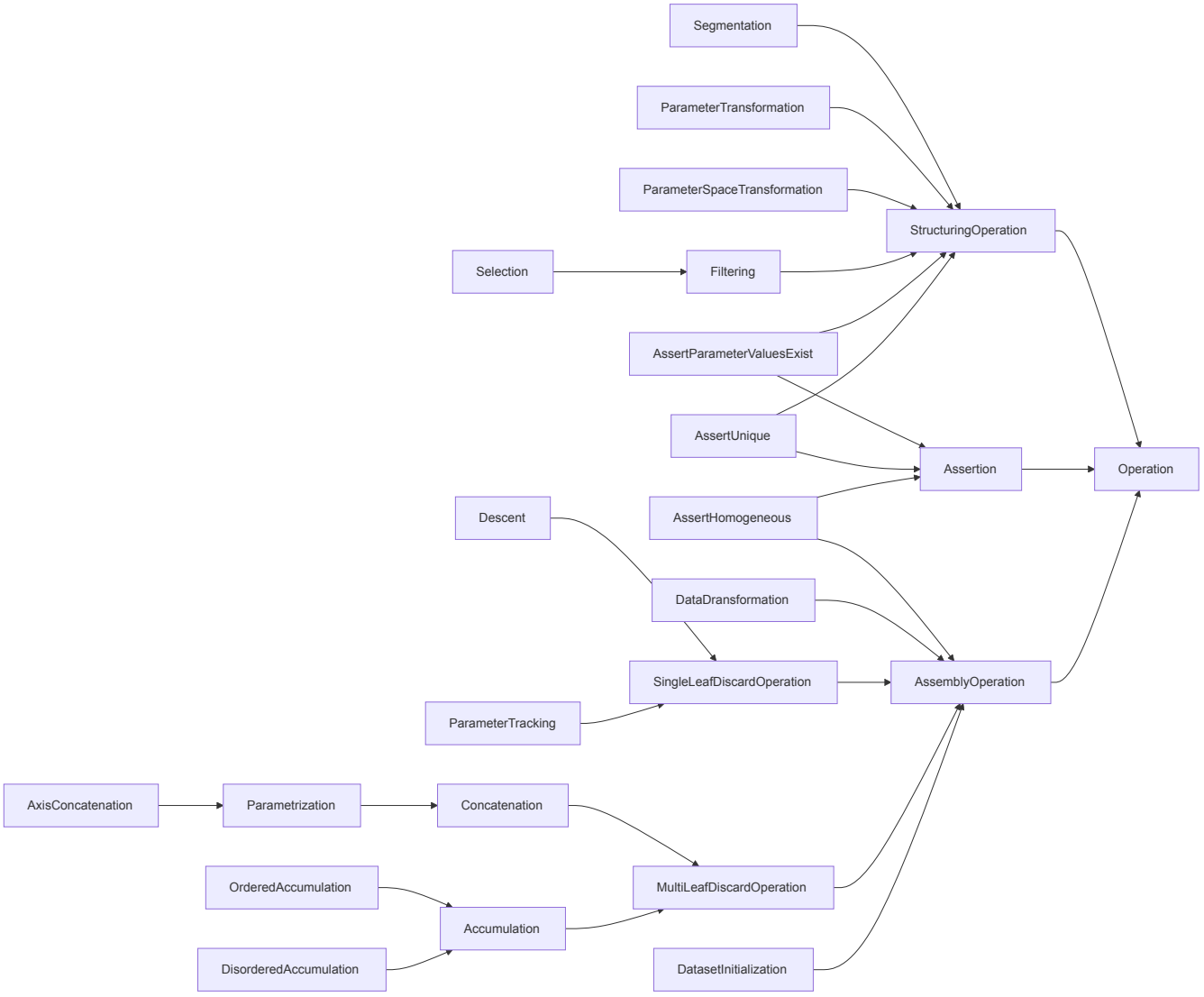
- Assertions
  - Parameter Value Exists
  - Unique Parameter

Pure Assembly Operations

- Data Transformation
- Single Leaf Discard Operations:
  - Descend (needs a new name)
  - Parameter Tracking / up-propagation
- Multi Leaf Combine Operations:
  - Parametrize
  - AsAxis (same as parametrize, but dataset must be homogeneous w.r.t. the axis)
  - Accumulate and Subclasses
- Assertions
  - dataset is homogenous (valid multi-dimensional array)

Though I don't have an example at hand at this time, there might also be Operations with one part performed on the way down and one on the way up.

A class structure could look something like this:


Segmentation
ParameterTransformation
ParameterSpaceTransformation
StructuringOperation
Selection
Filtering
AssertParameterValuesExist
AssertUnique
Assertion
Operation
Descent
AssertHomogeneous
DataDransformation
SingleLeafDiscardOperation
AssemblyOperation
ParameterTracking
AxisConcatenation
Parametrization
Concatenation
OrderedAccumulation
MultiLeafDiscardOperation
Accumulation
DisorderedAccumulation
DatasetInitialization

All these operations should have member functions named something like `on_descent()` and `on_ascent()` or `structure()/assemble()` or something like that… with well-defined signatures so they can be called from an outside class, the `Assembler` that doesn't know which specific type each operation has and what the implementations of their functions are.